

Pokhara University
Gandaki College of Engineering and Science
Lamachaur, Pokhara-16, Kaski



Lab 2: Implementing Different Features of an IDE (Visual Studio Code) - Build, Breakpoints, Refactoring, Source code collaboration (git), Palate code generation, IntelliSense

Submitted By:

Samika Khadka

BESE2023

Roll no. 33

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE

Implementation of Different Features of an IDE (Visual Studio Code) including Build, Breakpoints, Refactoring, Source Code Collaboration using Git, Palette Code Generation, and IntelliSense.

AIM

To explore and implement the core features offered by Visual Studio Code (VS Code) as an Integrated Development Environment (IDE), and to understand how these features improve developer productivity, code quality, and collaboration during software development.

THEORY

An Integrated Development Environment (IDE) is a software application that provides comprehensive facilities to programmers for software development. Visual Studio Code (VS Code) is a free, lightweight, and extensible source-code editor developed by Microsoft that supports multiple programming languages through extensions.

VS Code offers a rich set of built-in and extension-based features that streamline the development workflow. Some of the most widely used features are:

- Build – Compiling and running source code directly from the editor using build tasks or an integrated terminal.
- Breakpoints – Pausing program execution at specific lines during debugging to inspect variable states and program flow.
- Refactoring – Restructuring existing code (renaming variables, extracting methods, etc.) without changing its external behavior.

- Source Code Collaboration (Git) – Built-in Git integration for version control, allowing staging, committing, and pushing code directly from the editor.
- Palette Code Generation – Using the Command Palette (Ctrl+Shift+P) and code snippets to quickly generate boilerplate code and execute editor commands.
- IntelliSense – Smart code completion feature that provides suggestions for variables, methods, and syntax based on context.

Together, these features make VS Code a powerful and efficient IDE for individual and collaborative software development, supporting Agile practices such as continuous coding, testing, and version control.

OBJECTIVES

- Learn how to build and run programs within VS Code.
- Understand debugging using breakpoints.
- Perform code refactoring using VS Code tools.
- Use Git integration for source code collaboration.
- Use the Command Palette for quick code/task generation.
- Understand and use IntelliSense for efficient coding.

1. BUILD

Building refers to compiling and executing source code within the IDE using an integrated terminal or configured build tasks (tasks.json).

Algorithm

1. Open VS Code and create a new file, e.g., Sum.java or main.py.
2. Write the source code in the editor.
3. Save the file with the appropriate extension.

4. Open the Integrated Terminal (Ctrl+`).
5. Compile the code using the language-specific compiler command.
6. Run the compiled program.
7. Observe the output in the terminal.

Sample Program (Sum.java)

```
public class Sum {  
    public static void main(String[] args) {  
        int a = 15, b = 25;  
        int total = a + b;  
        System.out.println("Sum of " + a + " and " + b + " is: " + total);  
    }  
}
```

Build Commands

```
javac Sum.java  
java Sum
```

Output

```
Sum of 15 and 25 is: 40
```

2. BREAKPOINTS

A breakpoint is a marker set on a specific line of code that pauses program execution at that point during debugging, allowing inspection of variable values and program state.

Algorithm

1. Open the source file in VS Code.
2. Click on the left margin (gutter) next to the desired line number to set a breakpoint (a red dot appears).
3. Open the Run and Debug view (Ctrl+Shift+D).
4. Click 'Run and Debug' and select the appropriate environment.
5. Execution pauses when it reaches the breakpoint.
6. Inspect variable values in the Variables pane.
7. Use Step Over / Step Into / Continue to resume execution.

Sample Program (Factorial.java)

```
public class Factorial {  
    public static void main(String[] args) {  
        int n = 5;  
        int fact = 1;  
        for (int i = 1; i <= n; i++) {  
            fact = fact * i; // Breakpoint set on this line  
        }  
        System.out.println("Factorial of " + n + " is: " + fact);  
    }  
}
```

Output

```
Debugger paused at line 6 (breakpoint hit) — i = 1, fact = 1  
Debugger paused at line 6 (breakpoint hit) — i = 2, fact = 1  
Debugger paused at line 6 (breakpoint hit) — i = 3, fact = 2  
... (execution continues after inspection) ...  
Factorial of 5 is: 120
```

3. REFACTORING

Refactoring is the process of restructuring existing code without changing its external behavior, aimed at improving readability, maintainability, and structure. VS Code provides built-in refactoring tools such as Rename Symbol and Extract Method (accessible via right-click or Ctrl+.).

Algorithm

1. Open the source file containing the code to refactor.
2. Select the variable, method, or code block to refactor.
3. Right-click and choose 'Rename Symbol' (F2) or 'Refactor...' (Ctrl+.).
4. Choose the desired refactoring option (e.g., Extract Method, Rename).
5. Enter the new name or confirm the extraction.
6. VS Code automatically updates all references throughout the file/project.
7. Save the file and verify the program still runs correctly.

Before Refactoring

```
public class Calc {  
    public static void main(String[] args) {  
        int a = 10, b = 20;  
        int x = a + b;  
        System.out.println("Result: " + x);  
    }  
}
```

After Refactoring (Rename Symbol: x -> sumResult)

```
public class Calc {  
    public static void main(String[] args) {
```

```
int a = 10, b = 20;

int sumResult = a + b;

System.out.println("Result: " + sumResult);

}

}
```

Output

Result: 30

4. SOURCE CODE COLLABORATION (GIT)

VS Code has built-in Git integration accessible through the Source Control view (Ctrl+Shift+G), allowing developers to stage, commit, and push changes directly from the editor without using a separate terminal-based Git client.

Algorithm

1. Open the project folder in VS Code.
2. Open the Source Control panel (Ctrl+Shift+G).
3. Initialize a repository by clicking 'Initialize Repository' (or use terminal: `git init`).
4. Modify/create files; changes appear automatically under 'Changes'.
5. Stage the changes by clicking the '+' icon next to each file.
6. Enter a commit message and click the checkmark (✓) to commit.
7. Connect to a remote GitHub repository and click 'Sync/Push' to upload changes.

Commands (via Integrated Terminal)

```
git init
```

```
git add .  
git commit -m "Added Factorial and Calc programs"  
git remote add origin https://github.com/username/vscode-lab.git  
git branch -M main  
git push -u origin main
```

Output

```
Initialized empty Git repository in /project/.git/  
[main (root-commit) a1b2c3d] Added Factorial and Calc programs  
2 files changed, 20 insertions(+)  
Enumerating objects: 4, done.  
Writing objects: 100% (4/4), done.  
To https://github.com/username/vscode-lab.git  
* [new branch]    main -> main
```

5. PALETTE CODE GENERATION

The Command Palette (Ctrl+Shift+P) in VS Code provides quick access to all available commands, including generating configuration files, running tasks, and inserting code snippets. Combined with the built-in snippet system, it enables rapid boilerplate code generation.

Algorithm

1. Open VS Code and create/open a source file.
2. Press Ctrl+Shift+P to open the Command Palette.
3. Type a command, e.g., 'Java: Create Java Project' or 'Snippets: Insert Snippet'.
4. Select the desired snippet or command from the list.
5. VS Code automatically inserts the corresponding boilerplate code.

6. Modify the generated code as required and save the file.

Example: Generating a 'for' loop snippet (type 'for' + Tab)

```
// Typed: for<Tab> -> auto-generated by snippet
for (let i = 0; i < array.length; i++) {

}
```

Filled Program (main.js)

```
const array = [10, 20, 30, 40, 50];
let sum = 0;
for (let i = 0; i < array.length; i++) {
    sum += array[i];
}
console.log("Sum of array elements: " + sum);
```

Output

```
Sum of array elements: 150
```

6. INTELLISENSE

IntelliSense is a code-completion feature in VS Code that provides intelligent suggestions for variables, methods, function signatures, and syntax based on the language and current code context, reducing errors and speeding up development.

Algorithm

1. Open a source file in VS Code with the relevant language extension installed.
2. Start typing a variable, object, or method name.

3. Observe the automatic suggestion dropdown that appears (IntelliSense).
4. Use arrow keys to navigate suggestions.
5. Press Tab or Enter to auto-complete the selected suggestion.
6. Hover over a method/function to view its signature and documentation via IntelliSense.

Sample Program (String.java) — IntelliSense suggests .length(), .toUpperCase(), etc.

```
public class StringDemo {  
    public static void main(String[] args) {  
        String name = "vscode";  
        // Typing "name." triggers IntelliSense suggestions:  
        // length(), toUpperCase(), charAt(), substring(), etc.  
        System.out.println("Length: " + name.length());  
        System.out.println("Uppercase: " + name.toUpperCase());  
    }  
}
```

Output

```
Length: 6  
Uppercase: VSCODE
```

RESULT

The various features of Visual Studio Code as an IDE — Build, Breakpoints, Refactoring, Source Code Collaboration (Git), Palette Code Generation, and IntelliSense — were successfully explored and implemented with appropriate sample programs, and the expected outputs were obtained for each feature.

CONCLUSION

This experiment provided hands-on experience with the core features of Visual Studio Code as a modern IDE. It demonstrated how build tools, debugging with breakpoints, refactoring utilities, integrated Git-based version control, Command Palette code generation, and IntelliSense collectively enhance developer productivity, reduce coding errors, and support efficient, collaborative software development in line with Agile practices.